Software Dependability

Introduction to Software Vulnerabilities

What is a Software Vulnerability?

A **vulnerability** refers to the inability of a system to withstand the effects of a hostile environment.

A vulnerability is **an instance of a fault** in software specification, development, or configuration that can cause its execution to violate the (implicit or explicit) **security policy** defined within a software organization.

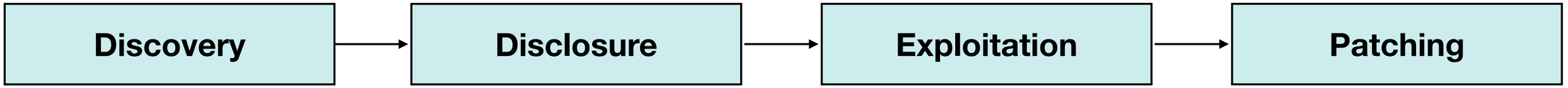
A vulnerability is a **weakness** that allows an external attacker to reduce a system's information assurance.

What is an Exploit?

An **exploit** means to take advantage of something for one's own end, especially unethically or unjustifiably.

An exploit is a **piece of software** that takes advantage of a bug or vulnerability to cause unintended behavior on computer software or hardware.

The Lifecycle of Software Vulnerabilities



Discovery. When a vendor, a hacker, or any third-party software analyst discovers a vulnerability.

Disclosure. A vulnerability is publicly disclosed, i.e., everyone is informed about it.

Exploitation. An external attacker can use the vulnerability to cause issues.

Patching. When a vendor fixes the vulnerability.

A Closer Look to Software Vulnerabilities: Buffer Overflow

Buffer overflow is one of the most common types of vulnerabilities.

```
/* Do something…*/
void function(){

  char dst[256];
  char* s = read_string();
  strcpy(dst, s);

}
```

It results from a **misuse of the string manipulation functions** in the standard C library.
If the length of the string is higher than 256 characters, the `strcpy()` function will write data over the available space.

A buffer is said to be overflown when the data (meant to be written into a memory buffer) **gets written past the left or right boundary of the buffer**. This way, the data gets written to a portion of memory that does not belong to the program variable that references the buffer.

Buffer overflows, if undetected, **can cause the program to crash or produce unexpected results**.

Why is Buffer Overflow so Problematic?

```c
#include <stdio.h>
#include <string.h>

int main(void) {

    char buff[15];
    int pass = 0;

    printf("\n Enter th
    gets(buff);

    if(strcmp(buff, “realpassword”)) {
        printf ("\n Correct Password \n”);
        pass = 1;
    } else {
        printf ("\n Wrong Password \n");
    }

    if(pass) {
       /* Now Give root or admin rights to user*/
       printf("\n Root privileges given to the user \n");
    }

    return 0;
}
```

the function does not check the array bounds

This piece of code simulates a scenario where a program expects a password from the user and grants root privileges to the user if the password is correct.

Let us try running the code:

```
$ ./bfrovrflw

 Enter the password:
 realpassword

 Correct Password

 Root privileges given to the user
```



```
$ ./bfrovrflw

 Enter the password :
 hhhhhhhhhhhhhhhhhhhh

 Wrong Password

 Root privileges given to the user
```



The attacker can supply an input of length greater than what the buffer can hold, even overwriting the memory of the integer ‘pass’ to let this value to be different than zero.

Despite the wrong password, the value of ‘pass’ is non-zero!

Types of Buffer Overflow

There are various similar forms of this vulnerability, which all result from missing input validations.

```
/* Do something…*/
void function(){
  char src[10], dst[10];
  char n = read_int();
  memcpy(dst, src, n);
}
```

Memcpy buffer overflow. If the value of n is bigger than the size of the destination buffer, the `memcpy()` function **will write data over the available space**.

```
/* Do something…*/
void function(){
  char s[256];
  int n = read_int();
  s[n] = “\0”;
}
```

Out-of-bounds array access. If the value of n is higher or lower than the size of the array, **it will be written over the available space**.

```
/* Do something…*/
void function(){
  char s[256];
  scanf(“%s”, s);
}
```

Scan buffer overflow. If the read value by the scanner is larger than 256 characters, **it will be written over the available spac**e.

SQL Injection

It is still one of the most common vulnerabilities in the wild.

```
https://insecure-website.com/products?category=Gifts
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

SQL injection is a web security vulnerability that **allows an attacker to interfere with database queries**.

It generally allows attackers to view data they should not be able to access, violating security, reliability, and sometimes availability.

- The query requests all details (*) from the ‘Products’ table that are categorized as ‘Gifts’ and released; i.e., the restriction excludes products not yet issued to the market.
- The application does not implement any defense against SQL injection attacks, so that an attacker might construct a query like:

```
https://insecure-website.com/products?category=Gifts’--‘

SELECT * FROM products WHERE category = ‘Gifts’--‘ AND released = 1
```

However, the double dash is a comment in SQL, so the rest of the query is ignored, allowing access to the non-released products too.

Cross-site Scripting

It is among one of the most common issues of Web apps.

```
<?php echo $_GET[‘id’]; ?>

www.samplewebsite.com/
```

This vulnerability allows an attacker **to “inject” client-side scripts into web application pages** with the ultimate aim of collecting, manipulating, or redirecting user information that should be private and protected.

If a website is vulnerable, exploiting the vulnerability and performing a cross-site scripting attack is relatively easy.

It is “just” needed to include some malicious code in the URL request.

```
<?php echo $_GET[‘id’]; ?>

www.samplewebsite.com/id=10
```

This command may show the 10th element of a list

```
<?php echo $_GET[‘id’]; ?>

www.samplewebsite.com/id=<script>alert(document.cookie);</script>
```

But what if something more is included?

Here the cookies are shown to the user

Broken Authentication and Session Management

It is a family of vulnerabilities that involve the risk that an attacker steals users' data.

```
http://example.com/sale/saleitems;jsessionid=2P0OC2JSNDLPSKHCJUN2JV?dest=Hawaii
```

The session ID can be directly modified!

These vulnerabilities allow attackers to **capture or bypass the authentication methods** used in a web application. Usually, these are due to:

1. User authentication credentials are not protected when stored.
2. Predictable login credentials.
3. Session IDs are exposed in the URL.
4. Session IDs are vulnerable to session fixation attacks.
5. Session values do not time out or get invalidated after logout.
6. Session IDs are not rotated after a successful login.
7. Passwords, session IDs, and credentials are sent over unencrypted connections.

Insecure Direct Object References

Similar to BASM, these vulnerabilities relate to the lack of control over the accesses of a web application.

```
https://insecure-website.com/customer_account?customer_number=132355
```

In many examples of access control vulnerabilities, user-controlled parameter values are used to directly access resources or functions.

In the example above, the customer number is used as a **record index in queries performed on the backend database**.

If no other controls are in place, an attacker can modify the `customer_number` value and bypass access controls to view other customers' records.

```
https://insecure-website.com/static/12144.txt
```

In this example, the website stores/accesses server-side resources using incrementally named filenames in the sent URL request.

An attacker can modify the filename to retrieve data created by other users and potentially obtain their credentials and sensitive data.

Security Misconfiguration

It is commonly the result of insecure default configurations, incomplete or ad hoc configurations, open cloud storage, misconfigured HTTP headers, and verbose error messages that contain sensitive information.

1. Appropriate security hardening is missing across the application stack, or **permissions are improperly configured** on cloud services.
2. **Unnecessary features are enabled or installed** (e.g., unnecessary ports, services, pages, accounts, or privileges).
3. Default accounts and their passwords are **still enabled and unchanged**.
4. Error handling reveals **stack traces or other overly informative error messages** to users.
5. For upgraded systems, **the latest security features are disabled or not configured**.
6. The **security settings** in application servers, frameworks (e.g., Struts, Spring), libraries, and databases **are not set to secure values**.
7. The server does not send **secure headers or directives**.
8. The software is **outdated or vulnerable**.

The Open Web Application Security Project (OWASP)

OWASP is a nonprofit foundation that works to improve software security. Through community-led open-source software projects, hundreds of local chapters worldwide, tens of thousands of members, and leading educational and training conferences, the OWASP Foundation is the source for developers and technologists to secure the web.

OWASP Top 10 is a standard awareness document for developers and web application security.

This list represents a broad consensus about the most critical security risks of web applications.

Link: https://owasp.org/www-project-top-ten/.

As for 2021, **broken access control, cryptographic failures, and injections** are included in the top-ten list.

More interestingly, the list includes the **“Vulnerable and Outdated Components”** problem.

Components, such as libraries, frameworks, and other software modules, run with the same privileges as the application.

Such an attack can result in severe data loss or a server takeover if a vulnerable component is exploited.

Applications and APIs using vulnerable components may undermine application defenses and enable various attacks and impacts.

=> This problem is growing because of the popularity and pervasiveness of open-source in all types of software projects!

The Common Vulnerabilities and Exposure (CVE)

CVE is the de facto standard dictionary that defines publicly disclosed cybersecurity vulnerabilities and exposures.

CVE is maintained by the MITRE Corporation, a nonprofit organization that operates R&D centers funded by the US federal government.

CVE aims at standardizing the names of all publicly known vulnerabilities and security exposures.

CVE aims to make it easier to share data across separate vulnerable databases and security tools.



1. **Creation.** The process of creating a new CVE entry starts with the discovery of a new potential security vulnerability.
2. **Verification.** The information is later assigned a CVE ID by a CVE Numbering Authority (CNA)
3. **Insertion.** The CNA writes the description and adds references, and then the completed CVE Entry is added to the CVE List and posted on the CVE website by the CVE Team.

You can signal new vulnerabilities!

The Common Vulnerabilities and Exposure (CVE)

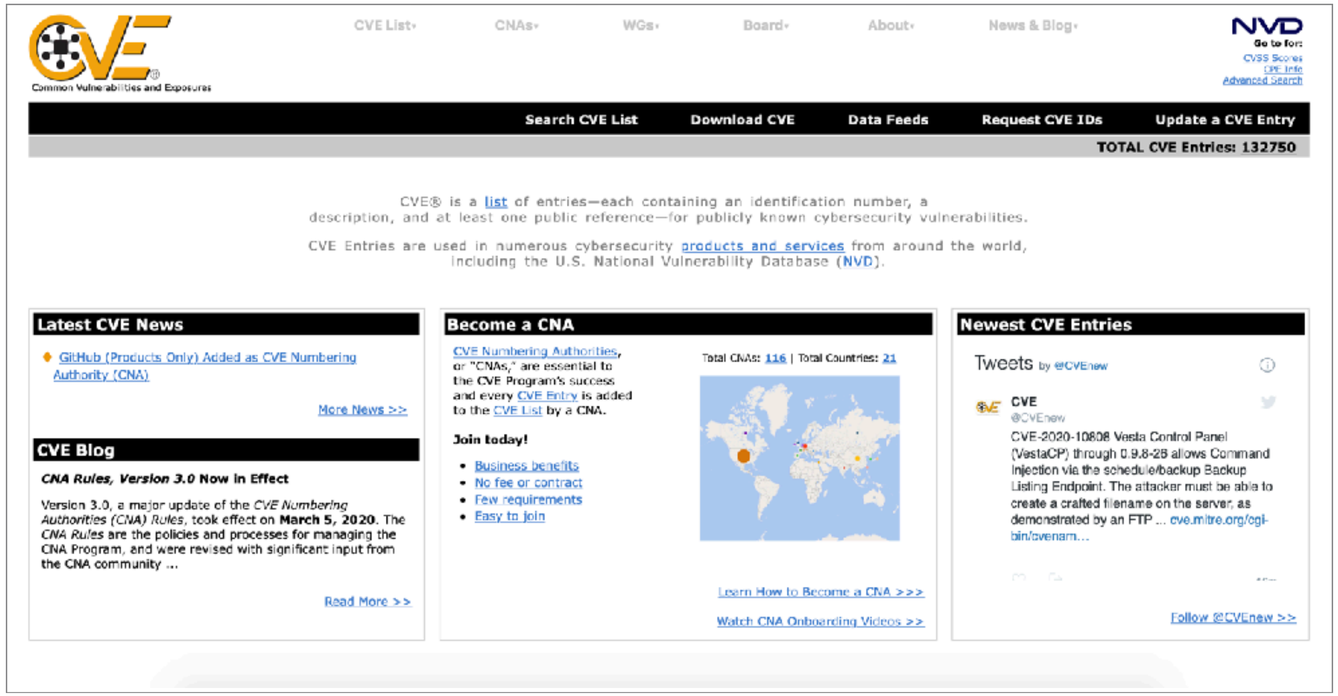
It is not just yet another database

The dictionary is designed to **link vulnerability databases and facilitate comparisons of security tools and services**.

However, it does not include information about their severity, potential impact, or detailed technical information about the CVEs.

NB: a CVE can be opened before exploitation or a resolution.

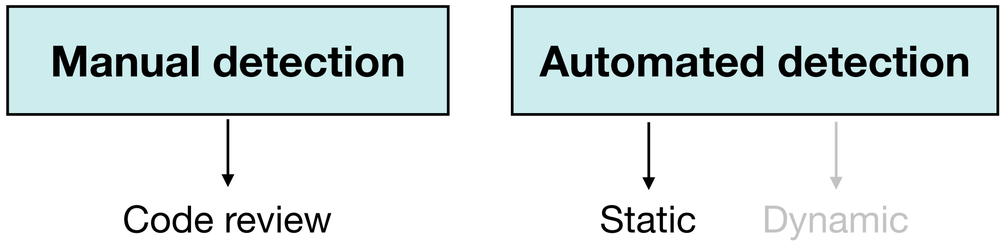
It means external attackers can exploit problems that are still not fixed using CVE information!



CVE is minable!
You can download all known vulnerabilities so far; this means you can access more than 100.

How to Detect Software Vulnerabilities?

The identification of vulnerabilities is tightly connected to testing.



Code review is the direct descendant of the most famous **code inspection**, a manual verification method to investigate code quality.

Code inspection refers to a **peer review** (i.e., done by developers for developers) of any artifact by trained individuals who look for defects and design issues using a well-defined process.

Let us have a quick look at the code review process.

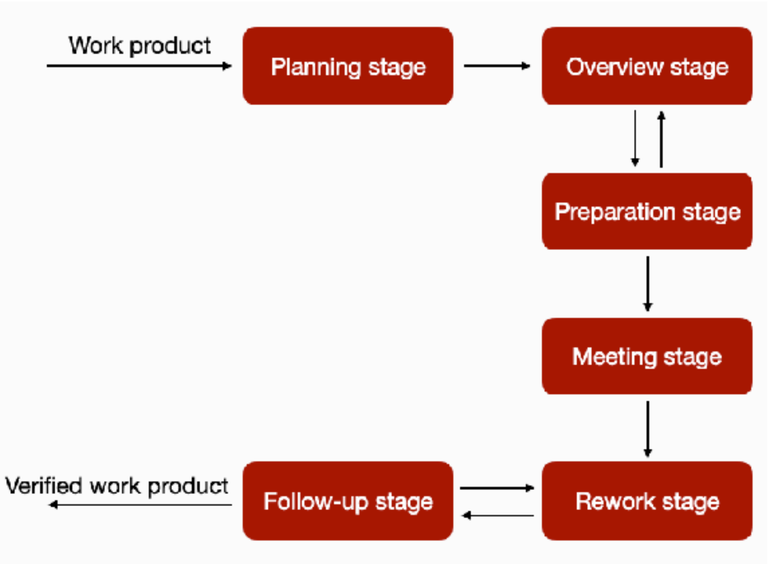
Finding Vulnerabilities through Code Review - Participants

- The **author** is the person who created the work product (artifact) being inspected. In pull request terms, they are the **contributor**.
- The **moderator** is the leader of the inspection. They aim to plan and coordinate the inspection - they are also **integrators**.
- The **inspector** is the person who raises questions, suggests problems, and criticizes the work product - they are called **reviewers**.
- The **recorder** documents the defects and design issues found during the code inspection process.

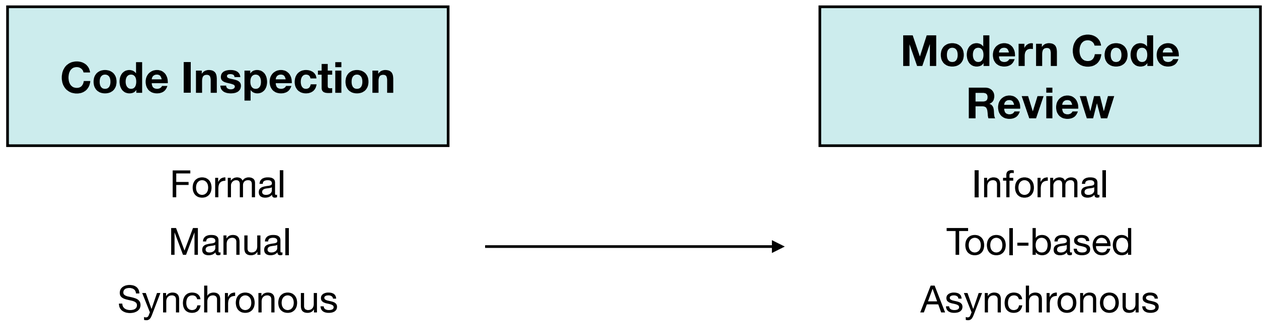
Finding Vulnerabilities through Code Review - Process

The process is formal, based on meetings where developers check the source code against a checklist reporting the aspects to verify.

If a problem is identified, the author of the modification must rework the code and then undergo another inspection.



From Traditional Code Inspection to Modern Code Review



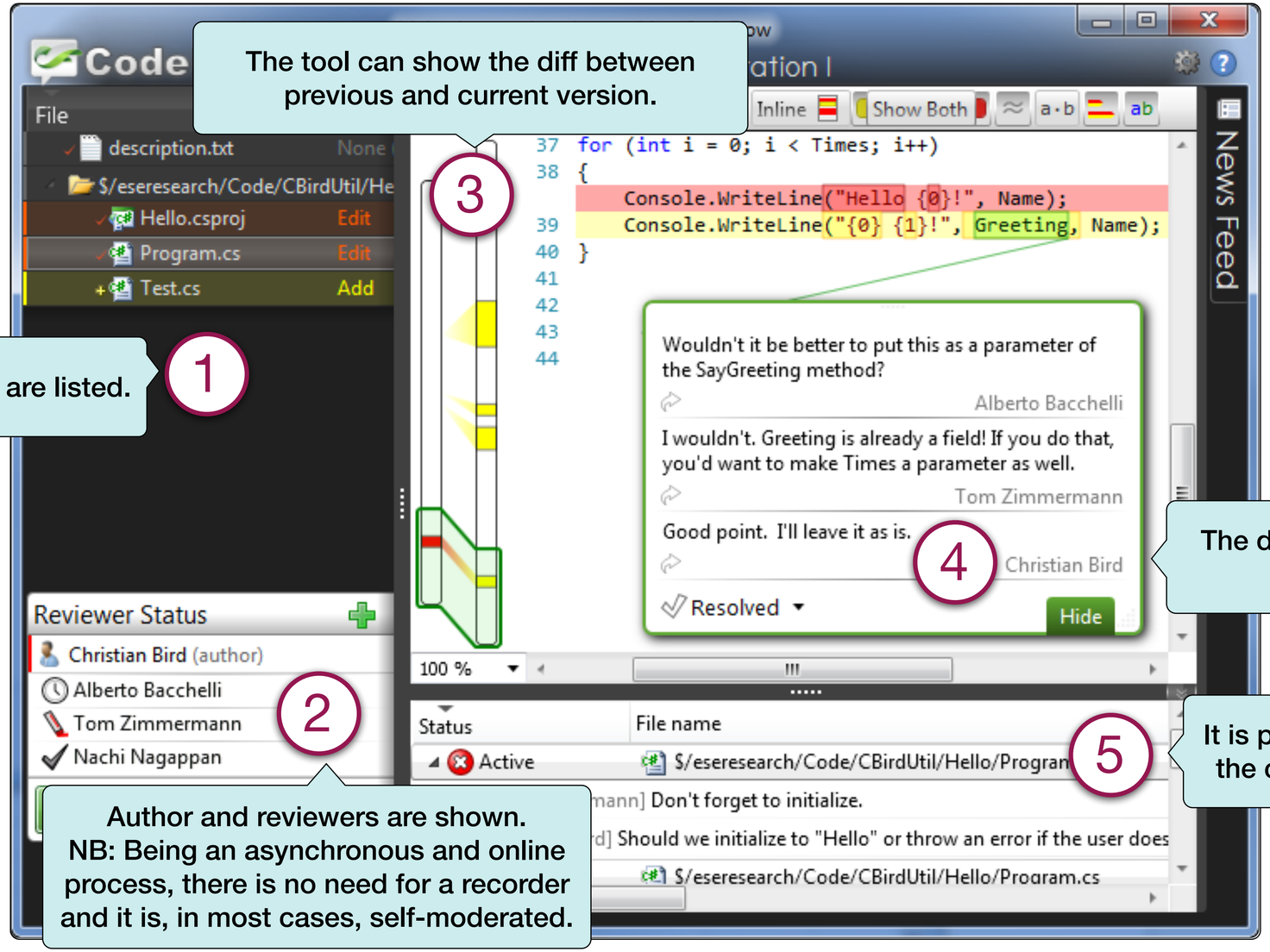
Modern code review (also known as **Contemporary Code Review**) represents a lightweight process that is (1) informal, (2) tool-based, (3) asynchronous, and (4) focused on inspecting new proposed code changes rather than the whole codebase.

In a typical code review process, developers (the **reviewers**) other than the code change author manually **inspect newly committed** changes to identify as many issues as possible and provide feedback that the change author must address before the code is accepted and put into production.

Modern code review is a **collaborative process** where reviewers and authors conduct an **asynchronous online discussion** to ensure that the proposed code changes are sufficiently high quality and fit the project's direction before being accepted.

In code reviews, discussions range from low-level concerns (e.g., variable naming) to high-level considerations (e.g., alignment with the scope of the project) and **encompass functional defects and evolutionary aspects**.

Microsoft's CodeFlow



A Software Security Checklist

1. Use query parameterization to prevent injection.
2. Scan your dependencies for known vulnerabilities.
3. Sanitize all input.
4. Avoid Java serialization.
5. Use strong encryption and hashing algorithms.
6. … any other violation you want to check.

Software Security is one of the most important non-functional requirements

Therefore, checklists are available and specifically studied to cover the most common vulnerabilities.

For instance, Oracle produced a list of secure coding guidelines:
https://www.oracle.com/java/technologies/javase/seccodeguide.html

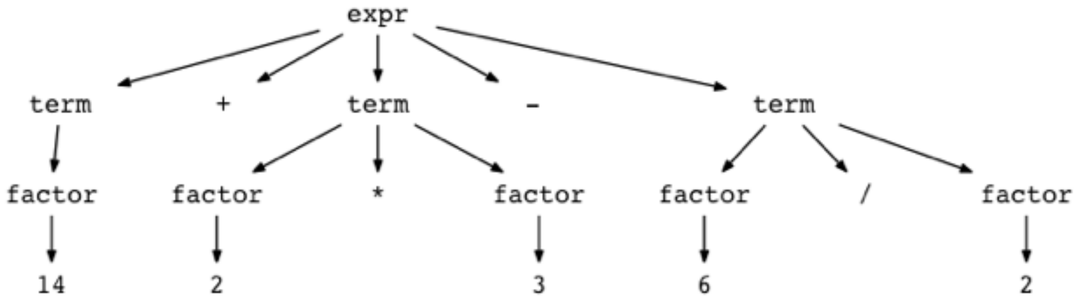
These checklists depend strongly on the type of application you analyze; e.g., a Web application is subject to different security threats than the software at the core of an autonomous car.

To return to the code review process, what is the main problem of code inspections?

Detecting Vulnerabilities using Static Analysis

Manual testing is extremely time-expensive and not repeatable.
However, some security guidelines can be automated through “simple” analyses of source code!

Let us consider this statement: $14 + 2 * 3 - 6/2$



Do you remember the abstract syntax tree?
Well, it is a first-class citizen in vulnerability detection.

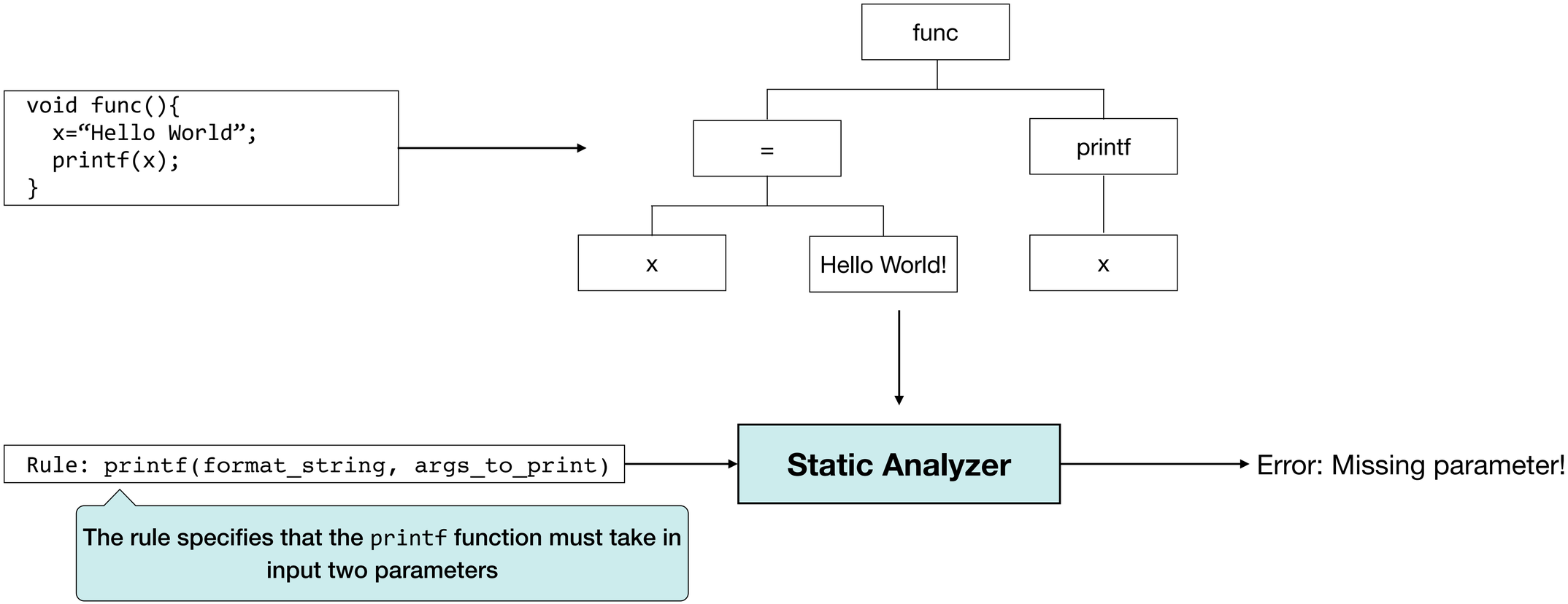
Static Analysis → Errors/violations

Security Ruleset

Static analysis takes as input an abstract syntax tree and a set of rules to check against it and identify violations of security rules.

Detecting Vulnerabilities using Static Analysis

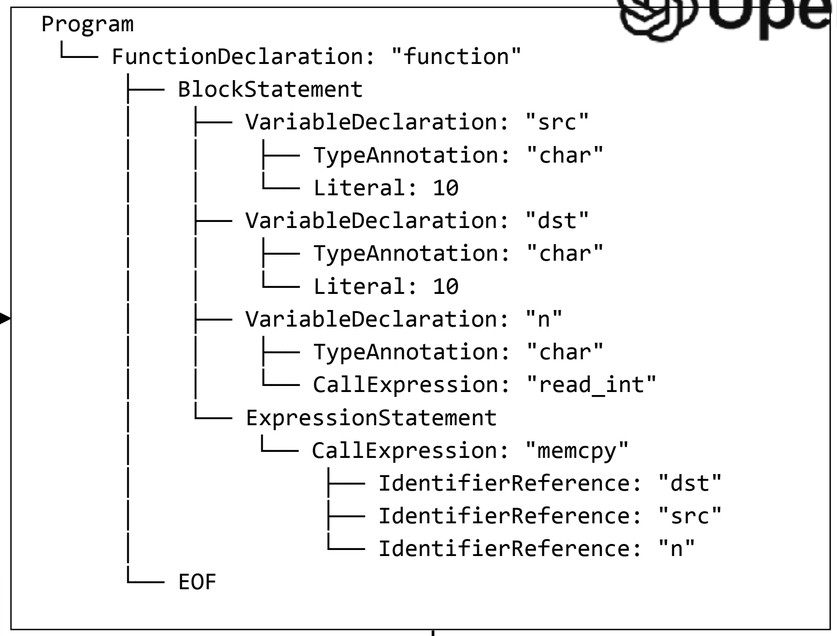
Let us consider a simple example.



Detecting Vulnerabilities using Static Analysis

Let us consider a simple example.

```
void function(){
  char src[10], dst[10];
  char n = read_int();
  memcpy(dst, src, n);
}
```



Rule:
S3649 36: SQL queries should not be vulnerable to injection attacks
S2076 5: OS commands should not be vulnerable to injection attacks
S2091 5: XPath expressions should not be vulnerable to injection attacks
S2078 5: LDAP queries should not be vulnerable to injection attacks
S2631 7: Regular expressions should not be vulnerable to Denial of Service attacks
S2083 7: I/O function calls should not be vulnerable to path injection attacks
…

sonarqube

FindSecBugs

Buffer Overflow!!!

Detecting Vulnerabilities using Static Analysis

Dozens of static analysis tools implement different security rules depending on the systems they analyze.

For example, SonarQube implements more than 50 static checks:
`https://sonarcloud.io/organizations/<your_organization_name>/rules?languages=java`

However, all that glitters is not gold!

- Most widely used static analyzers in industry, e.g., SonarQube, have low accuracy due to a high false-positive rate.
- Some rules require tuning thresholds: an endless problem. How to configure static analyzers effectively?
- Static analyzers are tools, not humans. They raise warnings based on a predefined ruleset that does not consider the specificities of the application being developed —> this further increases the risk of false positives.

How to do better?

First, we can leverage dynamic analysis, namely an analysis based on runtime information.
However, it is also possible to combine static and dynamic analyses!


Break